

Exercice 1

1. Ecrire le code des classes **Banque1** et **Banque2** contenant un constructeur par default et une méthode **void strategie ()** qui affiche le nom de la banque correspondante.
2. Ecrire le code de la classe **Guichet** contenant deux objets privés **B1** et **B2** de type **Banque1** et **Banque2**, respectivement, et une méthode **void action_effectuer(int)** qui fait appel à la stratégie de la banque (méthode strategie()) selon la valeur de l'entier reçu.
Exemple 123→stratégie de la banque 1
345→stratégie de la banque 2
stratégie de la banque 1 : par default
3. Ecrire un programme d'essai qui demande à l'utilisateur de saisir un entier et exécute la stratégie de la banque correspondante
4. Qu'il est le problème qu'on peut rencontrer avec cette conception.
5. Créer une classe abstraite **Banque_Strategie** contenant une seule méthode virtuelle pure **action_effectuer()**,
6. On souhaite que les classes **Banque1** et **Banque2** héritent de la classe **Banque_Strategie**, donner le code correspondant. Les deux classes doivent avoir une seule méthode chacune.
7. Créer la classe **Guichet** contenant un objet de type **Banque_Strategie ***, un constructeur par défaut, une méthode **setStrategie(Banque_strategy *c)** et une méthode **calcule()** qui fait appel à la méthode **action_effectuer ()**.
8. Ecrire un programme d'essai qui demande à l'utilisateur de saisir un entier et exécute la stratégie de la banque correspondante.

Exercice 2

En exploitant le code de l'exercice 6 de la 2ème série, on souhaite affecter à chaque joueur une arme convenable (Sniper→Fusil, Magicien→Baguette et Barbare→Marteau).

1. Ecrire le code de la classe **ArmeAbstraite** contenant un constructeur par default et une méthode virtuelle pure **void utiliser()**.
2. Ecrire le code des classes **Fusil**, **Marteau** et **Baguette** qui héritent publiquement de la classe **ArmeAbstraite** et redéfinissent la méthode **void utiliser()** en affichent le nom de l'arme.
3. Modifier la classe **Personnage** en lui ajoutant un attribut de type **ArmeAbstraite***.
2. écrire un programme qui crée, pour chaque joueur, l'arme qui lui convient. Le programme doit afficher l'arme utilisée par le joueur avant chaque attaque (avant appel des méthodes **frapper** et **attaque_spéciale**).

Exercice 3

1. Stocker les personnes créées dans l'exercice 4 (série numéro 2) dans un **vector**, **list** et **map**.
2. Afficher les données du **vector** et **map** en utilisant un itérateur sur un **vector** et sur une **map**.
3. Afficher les données de la **list** en utilisant l'algorithme **for_each** de STL.
4. Trier puis afficher les données des personnes en utilisant le **vector**.

Exercice4

1. Créez un bloc **try**, une instruction **catch** et une exception simple.
2. Modifiez la réponse précédente pour placer une donnée et une méthode d'accès dans l'exception et utilisez cette méthode dans le bloc **catch**.